

Introduction to Cryptography

Pseudorandom generators and the pseudo one-time pad

Lei Xu

lxu12@kent.edu

Computer Science Department

Kent State University

September 26, 2026

Warm-up: is this a PRG?

- Let $G(x) = x \parallel \text{parity}(x)$, where $\text{parity}(x) = x_1 \oplus \cdots \oplus x_n$ is the XOR of all n bits of x
- Which of the following proves that G is not a pseudorandom generator?
 - G is not expanding
 - Consider the following distinguisher D : $D(y)$ outputs 1 iff the first bit of y is 1
 - Consider the following distinguisher D : $D(y)$ outputs 1 iff the parity of the first n bits of y is 0
 - Consider the following distinguisher D : $D(y)$ outputs 1 iff the parity of the first n bits of y is equal to the last bit of y

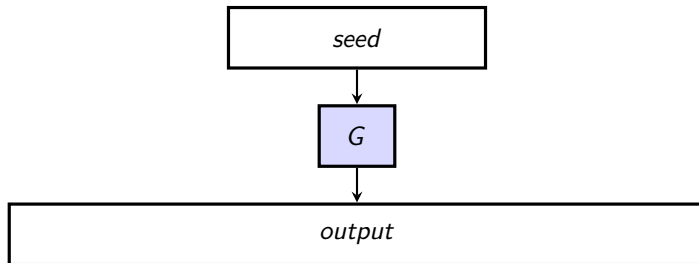
In this class

- *Pseudorandom generators*
 - A PRG stretches a short uniform seed into a longer output
 - That output distribution is far from uniform, yet no efficient distinguisher can tell
- *The pseudo one-time pad*
 - Replace the one-time pad's long key by $G(k)$, and prove security *by reduction*
 - If any efficient attacker breaks the scheme, that attacker distinguishes G from uniform
- *Toward multiple-message security*
 - The pseudo-OTP removes the long-key drawback, but not the one-time restriction
 - Keep the security goal, strengthen the threat model

Pseudorandom generators

PRGs

- Let G be a deterministic, poly-time algorithm that is *expanding*, i.e., $|G(x)| = p(|x|) > |x|$



Recall: what is a distribution?

- A distribution on ℓ -bit strings is a function $D : \{0, 1\}^\ell \rightarrow [0, 1]$ with $\sum_y D(y) = 1$
 - $D(y)$ is the probability that a sample equals y ; we write $y \leftarrow D$ for “sample y according to D ”
 - U_ℓ is the *uniform* distribution, assigning $2^{-\ell}$ to every ℓ -bit string
- Running a deterministic G on a *random* seed induces a distribution on its outputs

Example. Take $G(x) = x \parallel \text{parity}(x)$ with $n = 2$, so $x \leftarrow U_2$ and $G(x)$ is 3 bits:

$$G(00) = 000, \quad G(01) = 011, \quad G(10) = 101, \quad G(11) = 110$$

y	000	001	010	011	100	101	110	111
$\Pr[G(x) = y]$	$\frac{1}{4}$	0	0	$\frac{1}{4}$	0	$\frac{1}{4}$	$\frac{1}{4}$	0
$U_3(y)$	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{1}{8}$

- Every seed is equally likely, so each reachable output gets $1/4$ and the other four get 0
- At most 2^n of the $2^{p(n)}$ outputs are reachable, a fraction $2^{n-p(n)}$
 - Here that is exactly $1/2$; the larger the stretch $p(n) - n$, the smaller this gets

PRGs

- Let G be a deterministic, poly-time algorithm that is *expanding*, i.e., $|G(x)| = p(|x|) > |x|$
- G defines a sequence of distributions!
 - D_n = the distribution on $p(n)$ -bit strings defined by choosing $x \leftarrow U_n$ and outputting $G(x)$
 - $\Pr_{D_n}[y] = \Pr_{U_n}[G(x) = y] = \sum_{x: G(x)=y} \Pr_{U_n}[x] = \sum_{x: G(x)=y} 2^{-n} = |\{x : G(x) = y\}|/2^n$
 - Note that most y occur with probability 0
 - * I.e., D_n is far from uniform

Example. $G(x) = x \parallel \text{parity}(x)$ with $n = 2$:

- $\Pr_{D_2}[011] = |\{01\}|/2^2 = 1/4$ and $\Pr_{D_2}[001] = |\emptyset|/2^2 = 0$, while U_3 gives each $1/2^3 = 1/8$

PRGs

- G is a PRG iff $\{D_n\}$ is pseudorandom
- I.e., for all efficient distinguishers $\mathcal{A}$, there is a negligible function ϵ such that

$$\left| \Pr_{x \leftarrow U_n} [\mathcal{A}(G(x)) = 1] - \Pr_{y \leftarrow U_{p(n)}} [\mathcal{A}(y) = 1] \right| \leq \epsilon(n)$$

- I.e., no efficient $\mathcal{A}$ can distinguish whether it is given $G(x)$ (for uniform x) or a uniform string y !

Example 1 (insecure PRG)

- Let $G(x) = 0 \cdots 0$
 - Distinguisher?
 - Analysis?

Example 2 (insecure PRG)

- Let $G(x) = x \parallel \text{OR}(\text{bits of } x)$
 - Distinguisher?
 - Analysis?

Do PRGs exist?

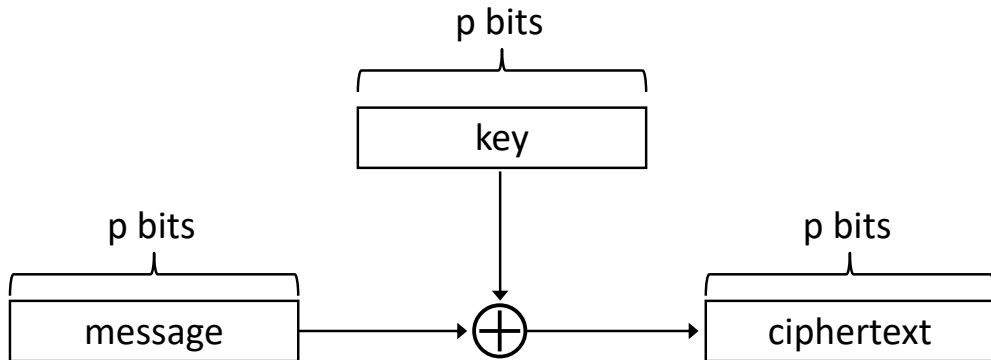
- We don't know. . .
 - Would imply $P \neq NP$
- We will *assume* certain algorithms are PRGs
 - Recall the 3 principles of modern crypto (Katz–Lindell): formal *definitions*, precise *assumptions*, *proofs* of security
 - Assuming “ G is a PRG” is the second principle at work: a single assumption, stated precisely and in isolation, that cryptanalysts can attack on its own
 - This is what is done in practice
- Can *construct* PRGs from weaker assumptions

Where things stand

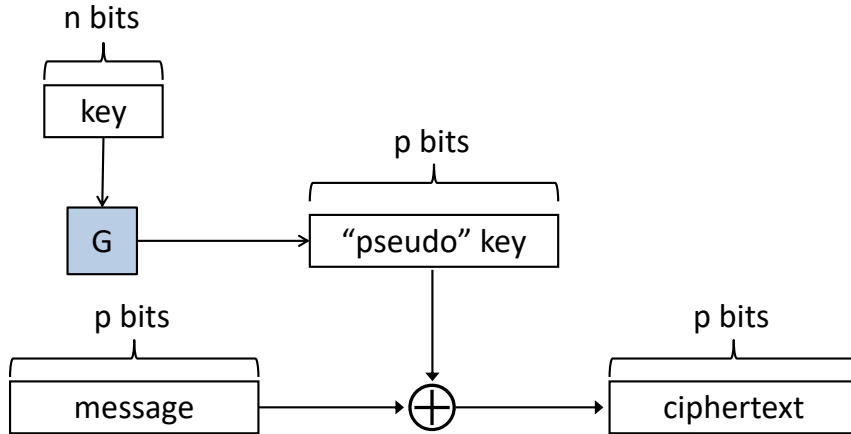
- We saw that there are some inherent limitations if we want perfect secrecy
 - In particular, key must be as long as the message
- We defined computational secrecy, a relaxed notion of security
- Can we overcome prior limitations with the relaxed notion of security?

The pseudo one-time pad

Recall: one-time pad



“Pseudo” one-time pad



Pseudo one-time pad

- Let G be a deterministic algorithm, with $|G(k)| = p(|k|)$
- $\text{Gen}(1^n)$: output uniform n -bit key k
 - Security parameter $n \implies$ message space $\{0, 1\}^{p(n)}$
- $\text{Enc}_k(m)$: output $G(k) \oplus m$
- $\text{Dec}_k(c)$: output $G(k) \oplus c$
- Correctness is obvious. . .

Key change

$G(k)$ is used to replace a long uniform key

Security of pseudo-OTP?

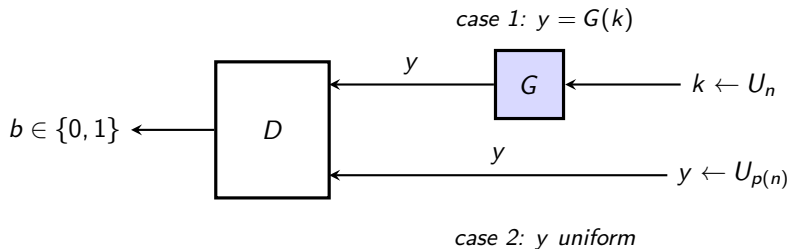
- Would like to be able to *prove* security
 - Based on the *assumption* that G is a PRG

Definitions, proofs, and assumptions

- We've *defined* computational secrecy
- Our goal is to *prove* that the pseudo OTP meets that definition
- We cannot prove this unconditionally
 - Beyond our current techniques. . .
 - Anyway, security clearly depends on G
- *Can* prove security based *on the assumption* that G is a pseudorandom generator

PRGs, revisited

- Let G be an efficient, deterministic function with $|G(k)| = p(|k|)$



- For any efficient D , the probabilities that D outputs 1 in each case must be “close”

Proof by reduction: by contradiction

- Assume G is a pseudorandom generator
- Assume toward a contradiction that there is an efficient attacker $\mathcal{A}$ who “breaks” the pseudo-OTP scheme (as per the definition)
- Use $\mathcal{A}$ as a subroutine to build an efficient D that “breaks” pseudorandomness of G
 - By assumption, no such D exists!
 - $\implies$ No such $\mathcal{A}$ can exist

Proof by reduction: direct bound

- Assume G is a pseudorandom generator
- Fix some arbitrary, efficient $\mathcal{A}$ attacking the pseudo-OTP scheme
- Use $\mathcal{A}$ as a subroutine to build an efficient D attacking G
 - Relate the distinguishing gap of D to the success probability of $\mathcal{A}$
- By assumption, the distinguishing gap of D must be negligible
 - $\implies$ Use this to bound the success probability of $\mathcal{A}$
- **This is the formulation we follow below**
 - Nothing is assumed about $\mathcal{A}$ beyond efficiency — no contradiction is needed
 - The result is a *bound* on $\mathcal{A}$'s success probability, holding for every efficient $\mathcal{A}$

Security theorem

Theorem

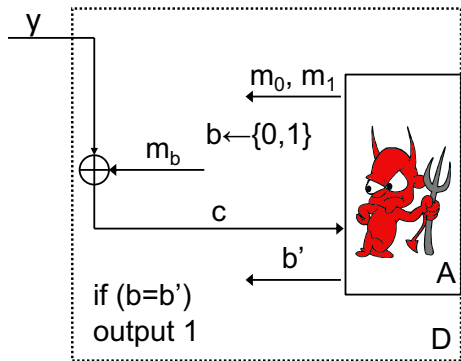
If G is a pseudorandom generator, then the pseudo one-time pad Π is computationally indistinguishable.

- Recall what that conclusion says — the experiment $\text{PrivK}_{\mathcal{A},\Pi}(n)$:
 - $\mathcal{A}(1^n)$ outputs m_0, m_1 of equal length
 - $k \leftarrow \text{Gen}(1^n)$, $b \leftarrow \{0, 1\}$, $c \leftarrow \text{Enc}_k(m_b) = G(k) \oplus m_b$
 - $b' \leftarrow \mathcal{A}(c)$
 - The experiment evaluates to 1 exactly when $b = b'$, i.e. when $\mathcal{A}$ guesses which message was encrypted
- Π is *computationally indistinguishable* if for every PPT $\mathcal{A}$ there is a negligible ϵ with

$$\Pr[\text{PrivK}_{\mathcal{A},\Pi}(n) = 1] \leq \frac{1}{2} + \epsilon(n)$$

- Guessing at random already achieves $\frac{1}{2}$, so the claim is that $\mathcal{A}$ does no better by more than $\epsilon(n)$

The reduction



D is given a $p(n)$ -bit string y , and runs $\mathcal{A}$ as a subroutine:

1. $\mathcal{A}$ outputs two messages m_0, m_1
2. D chooses a bit $b \leftarrow \{0, 1\}$
3. D hands $\mathcal{A}$ the ciphertext $c = y \oplus m_b$
4. $\mathcal{A}$ outputs a guess b'
5. D outputs 1 iff $b = b'$

- D uses the string y it was handed as *the pad*, so it never needs a key of its own
- D outputs 1 exactly when $\mathcal{A}$'s guess is correct — and $\mathcal{A}$ is never told which case it is in, i.e. whether y is $G(k)$ or uniform

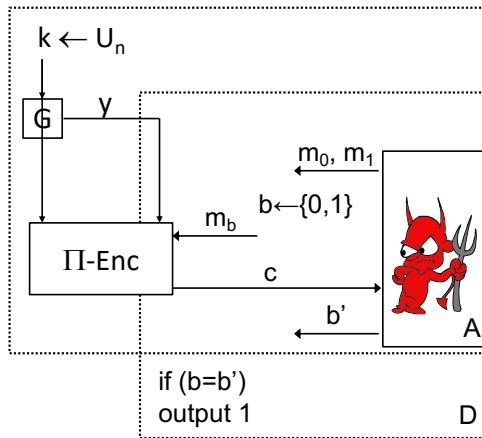
Analysis

- If $\mathcal{A}$ runs in polynomial time, then so does D
 - A PPT calls another PPT is still a PPT

Analysis

- Let $\mu(n) = \Pr[\text{PrivK}_{\mathcal{A},\Pi}(n) = 1]$
- Claim: when $y = G(x)$ for uniform x , then the view of $\mathcal{A}$ is *exactly* as in $\text{PrivK}_{\mathcal{A},\Pi}(n)$
 - $\implies \Pr_{x \leftarrow U_n}[D(G(x)) = 1] = \mu(n)$

The reduction: $y = G(k)$

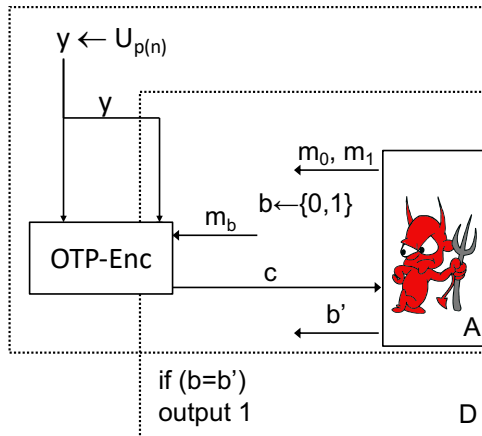


Case 1: $y = G(k)$ for uniform k . Inside D , $\mathcal{A}$ sees exactly the pseudo-OTP experiment $\text{PrivK}_{\mathcal{A}, \Pi}(n)$, so D outputs 1 with probability $\mu(n)$.

Analysis

- Let $\mu(n) = \Pr[\text{PrivK}_{\mathcal{A},\Pi}(n) = 1]$
- If $y = G(x)$ for uniform x , then the view of $\mathcal{A}$ is *exactly* as in $\text{PrivK}_{\mathcal{A},\Pi}(n)$
 - $\implies \Pr_{x \leftarrow U_n}[D(G(x)) = 1] = \mu(n)$
- If distribution of y is uniform, then $\mathcal{A}$ succeeds with probability exactly $\frac{1}{2}$
 - $\implies \Pr_{y \leftarrow U_{p(n)}}[D(y) = 1] = \frac{1}{2}$

The reduction: y uniform



Case 2: $y \leftarrow U_{p(n)}$. Inside D , $\mathcal{A}$ sees a true one-time pad ciphertext, so its guess is correct — and D outputs 1 — with probability exactly $\frac{1}{2}$.

Analysis

- Let $\mu(n) = \Pr[\text{PrivK}_{\mathcal{A},\Pi}(n) = 1]$
- If $y = G(x)$ for uniform x , then the view of $\mathcal{A}$ is *exactly* as in $\text{PrivK}_{\mathcal{A},\Pi}(n)$
 - $\implies \Pr_{x \leftarrow U_n}[D(G(x)) = 1] = \mu(n)$
- If distribution of y is uniform, then $\mathcal{A}$ succeeds with probability exactly $\frac{1}{2}$
 - $\implies \Pr_{y \leftarrow U_{p(n)}}[D(y) = 1] = \frac{1}{2}$
- Since G is pseudorandom, D 's gap is negligible: $|\mu(n) - \frac{1}{2}| \leq \text{negl}(n)$
 - Expanding the absolute value: $-\text{negl}(n) \leq \mu(n) - \frac{1}{2} \leq \text{negl}(n)$
 - Equivalently, $\frac{1}{2} - \text{negl}(n) \leq \mu(n) \leq \frac{1}{2} + \text{negl}(n)$
 - Security needs only the upper half: $\Pr[\text{PrivK}_{\mathcal{A},\Pi}(n) = 1] \leq \frac{1}{2} + \text{negl}(n)$
 - The lower half is no restriction — $\mathcal{A}$ can always reach $\frac{1}{2}$ by guessing b at random

Stepping back...

- *Proof* that the pseudo OTP is secure...
 - We have a provably secure scheme, rather than just a heuristic construction!

Stepping back...

- *Proof* that the pseudo OTP is secure...
- ...with some caveats
 - Assuming G is a pseudorandom generator
 - Relative to our definition
- The *only* ways the scheme can be broken are:
 - If a weakness is found in G
 - If the definition isn't sufficiently strong...

Have we gained anything?

- YES: the pseudo-OTP has a key shorter than the message
 - n bits vs. $p(n)$ bits
- The fact that the parties *internally* generate a $p(n)$ -bit temporary string to encrypt/decrypt is **irrelevant**
 - The *key* is what the parties share *in advance*
 - Parties do not store the $p(n)$ -bit temporary value

Toward multiple-message security

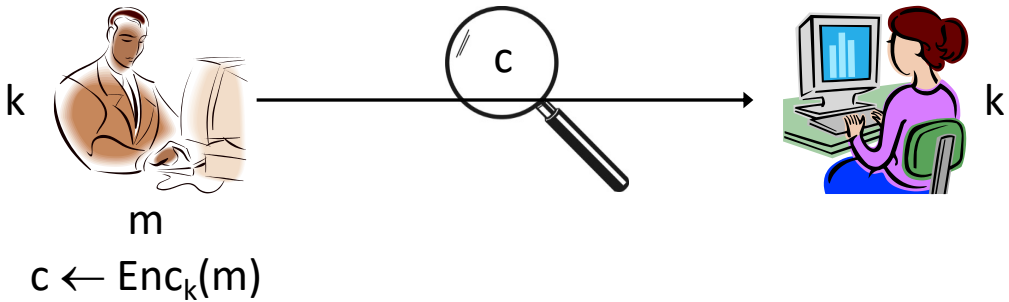
Recall. . .

- Perfect secrecy has two limitations/drawbacks
 - Key as long as the message
 - Key can only be used once
- We have seen how to circumvent the first
- Does the pseudo OTP have the second limitation?
- How can we circumvent the second?

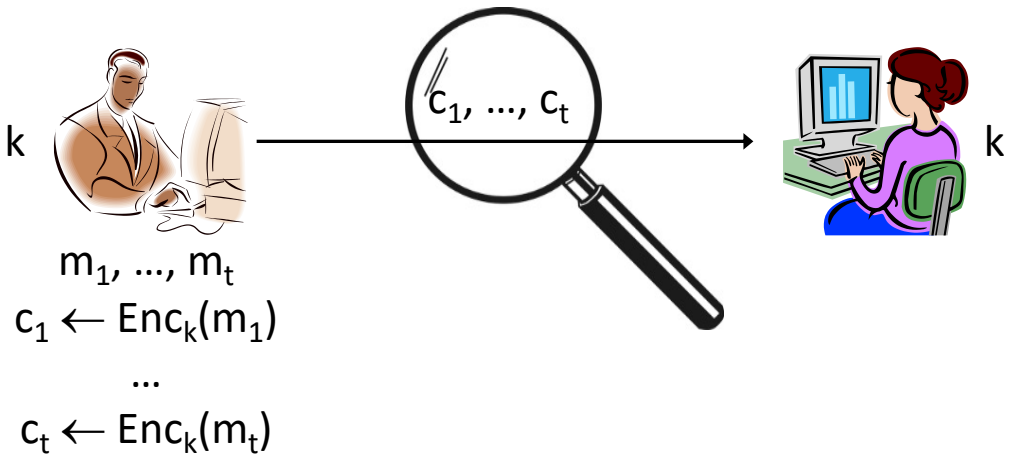
But first. . .

- Develop an appropriate security definition
- Recall that security definitions have two parts
 - Security goal
 - Threat model
- We will keep the security goal the same, but strengthen the threat model

Single-message secrecy



Multiple-message secrecy



A formal definition

- Fix $\Pi, \mathcal{A}$
- Define a randomized exp't $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{mult}}(n)$:
 - $\mathcal{A}(1^n)$ outputs two **vectors** $(m_{0,1}, \dots, m_{0,t})$ and $(m_{1,1}, \dots, m_{1,t})$
 - * Require that $|m_{0,i}| = |m_{1,i}|$ for all i
 - $k \leftarrow \text{Gen}(1^n), \quad b \leftarrow \{0, 1\}, \quad \text{for all } i: c_i \leftarrow \text{Enc}_k(m_{b,i})$
 - $b' \leftarrow \mathcal{A}(c_1, \dots, c_t); \quad \mathcal{A} \text{ succeeds if } b = b', \text{ and experiment evaluates to 1 in this case}$

A formal definition

- Π is *multiple-message indistinguishable* if for all PPT attackers $\mathcal{A}$, there is a negligible function ϵ such that $\Pr[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{mult}}(n) = 1] \leq \frac{1}{2} + \epsilon(n)$
- Exercise: show that the pseudo-OTP is *not* multiple-message indistinguishable